

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Elaborato d'esame

Fondamenti di Reti di Calcolatori

Raccolta e analisi di tracce di traffico di applicazioni mobili

---

Anno Accademico 2022/2023

Professore

Prof. Antonio Pescapè

Gruppo

Bruno Ruggiero
N46005787

Luciano Vitale
N46004566

Federica Sarnataro
N46005577

Francesco Tafuri
N46005941

Indice

Introduzione 2

Capitolo 1: Traffico e contestualizzazione del problema 3

Capitolo 2: Strumenti Utilizzati 4

Capitolo 3: Catture e analisi del traffico 6

Capitolo 4: Risultati sperimentali 9

Conclusioni 17

Introduzione

Il seguente elaborato ha lo scopo di raccogliere e analizzare tracce di traffico di applicazioni mobili. Ciò è stato possibile grazie agli strumenti messi a disposizione dalla Facoltà di Ingegneria della Federico II, con sede a Via Claudio (Napoli). Il lavoro è partito inizialmente da una fase di cattura del traffico svolta in laboratorio e successivamente, in separata sede, abbiamo analizzato il tutto tramite l'utilizzo di software specifici.
Di fondamentale importanza è stato il percorso seguito in aula che ci ha permesso di comprendere ed elaborare i risultati ottenuti oltre a perfezionare le singole velleità accademiche degli studenti coinvolti.

Capitolo 1: Traffico e contestualizzazione del problema

Con l'avanzare della tecnologia e con lo sviluppo di smartphone e altri dispositivi, che hanno rivoluzionato le nostre abitudini, anche il traffico di rete ha subito una notevole mutazione. Al giorno d'oggi è difficile considerare le innumerevoli azioni quotidiane distaccandosi dal concetto di traffico, si pensi ad esempio alla comunicazione tra persone o alla semplice azione di acquisto di un prodotto. Oltre all'evoluzione del traffico di rete è stata necessaria un'evoluzione anche per quanto riguarda le tecniche utili per capire il tipo di informazioni che vi transitano.

Classificare il traffico mobile fornisce informazioni utili in più ambiti come quello della pubblicità o della sicurezza, ma porta con sé una serie di problematiche legate alla sensibilità delle informazioni e della privacy.

Classificare il traffico vuol dire associare flussi di dati alle app che li generano, facendo ciò possiamo capire quali protocolli viaggiano su una rete.
Esistono varie tecniche di classificazione, distinguiamo alcune delle principali:

Port based: usa le porte assegnate a ciascun protocollo di default, classifica il traffico associando porta/protocollo.

Packet based: ogni pacchetto viene analizzato singolarmente, indipendente dagli altri.

Flow based: un flusso viene definito come una quintupla (Source IP address, Destination IP address, Source Port, Transport Protocol ID) e per la classificazione si utilizzano le informazioni estratte da un singolo pacchetto.

Payload based: il DPI (Deep Packet Inspection) si basa su questo metodo di classificazione. Con questa tecnica si dà una classificazione del protocollo analizzando parte dei dati del pacchetto e ciò avviene tramite specifiche caratteristiche del protocollo, confrontate e analizzate tramite dissectors.

Classificare il traffico in ambito mobile è sicuramente un argomento più complesso visto l'aumento dell'utilizzo di protocolli cifrati, il numero di app tra cui distinguere e la dinamicitá e l'eterogeneità di questo tipo di traffico.

Capitolo 2: Strumenti Utilizzati

Per effettuare il lavoro di cattura sono stati utilizzati gli strumenti forniti dall'ARC LAB della Federico II, trattasi di: due smartphone su cui sono stati sbloccati i permessi di root con installata la custom rom Lineage OS, la stazione di cattura costituita da un computer sul quale era eseguito il programma di cattura MIRAGE sviluppato dalla Federico II, due cavi usb per l'interfacciamento dei dispositivi mobile con il sistema di cattura. Una volta effettuata l'attività di cattura del traffico dati, si è proceduto all'analisi in locale dei dati grezzi mediante l'ausilio dei seguenti software:

Whois: è un protocollo di rete che consente, mediante queries di appositi database server (nei quali vengono raccolte le informazioni relative ai titolari dei nomi a dominio) da parte di un client, di stabilire a quale provider internet appartenga un determinato indirizzo IP o uno specifico DNS. Si può utilizzare tradizionalmente da riga di comando.

Capinfos: è un programma che legge uno o più file di cattura e restituisce statistiche e informazioni generali sulla cattura, come il numero di pacchetti spedito, la durata complessiva e il rate per lo scambio di pacchetti. L'output può essere ottenuto in due tipi di formati: lungo o tabella. L'utente specifica quale tipo di output (lungo o tabella) e quali statistiche visualizzare specificando flag (opzioni) che corrispondono al tipo di report e alle informazioni desiderate. Se non viene specificata alcuna opzione, Capinfos riporterà tutte le statistiche disponibili in formato di default "lungo".
[Per ulteriori informazioni, vedi manuale ufficiale]

TShark: è un analizzatore di protocollo di rete. Consente di acquisire i dati dei pacchetti da una rete live, di leggere i pacchetti da una cattura salvata precedentemente, stampando una forma decodificata di questi pacchetti su terminale o su di un file. Se non viene specificata alcuna opzione, TShark utilizzerà la libreria pcap per acquisire il traffico dalla prima interfaccia di rete disponibile e mostrerà una riga di riepilogo per ogni pacchetto ricevuto.
[Per ulteriori informazioni, vedi manuale ufficiale]

Di seguito le principali opzioni utilizzate:

- **-r:** Aggiungendo questa opzione possiamo specificare un file di acquisizione da cui TShark leggerà i pacchetti e rilascerà una riga di riepilogo per ogni pacchetto letto. Questo è un utilizzo di tipo statico del tool, dove è necessario avere un file pcap da leggere.

- **-e:** Aggiunge un campo all'elenco dei campi da visualizzare se -T fields è selezionato.

- **-z conv,type[,filter]:** Crea una tabella che elenca tutte le conversazioni che potrebbero essere visualizzate nell'acquisizione. Type specifica i tipi di endpoint di conversazione per i quali vogliamo generare le statistiche (ad esempio TCP o UDP). Questa opzione è stata utilizzata per estrarre i biflussi TCP e UDP, generando le tabelle riportate nel paragrafo di analisi.

- **-Y:** Con questa opzione è possibile abilitare un filtro che permette di visualizzare solamente i pacchetti che superano il filtro. I pacchetti che non supereranno il filtro non verranno presi in considerazione.

Capitolo 3: Catture e analisi del traffico

La prima fase dell' attività di laboratorio riguarda la cattura del traffico online. In particolare ci siamo organizzati in due gruppi di lavoro, ognuno formato da due persone:

- la prima persona si è recata in laboratorio e, con l'aiuto dei ricercatori presenti, ha avviato MIRAGE: un software gestito dall'università, progettato appositamente per catturare traffico di rete di app mobile.
  Successivamente sono stati collegati al PC, dove girava il sistema di cattura, due dispositivi mobile e da questi sono state avviate varie applicazioni di messaggistica, audio e video call.

- La seconda persona si è connessa da remoto, con il proprio dispositivo mobile, col referente in laboratorio. (Al fine di ottenere un'analisi sensata del traffico è importante che la seconda persona non sia connessa alla stessa rete Wi-Fi del suo compagno di lavoro);

Ogni gruppo ha eseguito 16 sessioni di catture, per un totale di 32 sessioni. Ogni sessione era composta da due attività, alternate tra chat, audio call e video call, dalla durata di 3 minuti l'una. Il tempo di attesa tra due sessioni e tra due attività era di 1 minuto.

Di seguito si riporta l'elenco delle sessioni svolte:

PRIMO GRUPPO
Studente Vitale Luciano situato in laboratorio ARC LAB.
Studente Ruggiero Bruno situato a Bologna.
Il gruppo ha eseguito le seguenti sessioni di attività, mediante le app Discord e Google Meet.

DISCORD

1. chat - audio call
2. chat - audio call
3. chat - video call
4. chat - video call
5. audio call - video call
6. audio call - video call
7. audio call - video call
8. audio call - video call

GOOGLE MEET

1. audio call - video call
2. audio call - video call
3. audio call - video call
4. audio call - video call
5. video call - audio call
6. video call - audio call
7. video call - audio call
8. video call - audio call

SECONDO GRUPPO

Studente Tafuri Francesco situato in laboratorio ARC LAB.
Studentessa Sarnataro Federica situata a Quarto (NA).
Il gruppo ha eseguito le seguenti sessioni di attività, mediante le app TrueConf e Jitsi Meet.

TRUECONF

1. video call - chat
2. video call - chat
3. audio call - chat
4. audio call - chat
5. video call - audio call
6. video call - audio call
7. audio call - video call
8. audio call - video call

JITSI MEET

1. audio call - video call
2. audio call - video call
3. audio call - video call
4. audio call - video call
5. video call - audio call
6. video call - audio call
7. video call - audio call
8. video call - audio call

Una volta terminate le sessioni, i risultati delle catture sono stati memorizzati sui propri dispositivi di massa (necessari almeno 4GB di memoria) in modo tale da poterli elaborare successivamente.
Abbiamo poi eseguito, per ciascuna cattura (un file "traffic.pcap") le seguenti operazioni:

- Estrazione delle informazioni preliminari e generali.
- Estrazione dei biflussi delle connessioni TCP e UDP.
- Analisi del traffico (DNS, SNI di TLS e host HTTP).
- Ricerca di metadati sulla compagnia proprietaria dell'indirizzo IP.

Per automatizzare l'analisi delle catture ci siamo serviti dello script bash in allegato (l'analisi è stata eseguita su sistema operativo Linux).

Per utilizzarlo è necessario, prima di tutto, collocarlo nella directory in cui sono presenti i file da analizzare (le tracce .pcap) e ,una volta aperto il terminale nella stessa directory, digitare il comando per eseguire lo script (./nome script.sh nome traccia.pcap).

Di seguito viene spiegato il funzionamento dello stesso:

La prima istruzione utilizza il comando Capinfos e salva i risultati in un file INFO.csv. Se il file non esiste viene creato nel momento in cui si esegue l'istruzione.
La seconda e la terza istruzione aggiungono al file creato in precedenza informazioni in formato tabellare sui flussi bidirezionali: sia in TCP che in UDP.
Con le istruzioni successive abbiamo poi ricavato le informazioni relative al DNS, al campo SNI di TLS e agli host HTTP. Una volta estratte, tutte le informazioni sono state aggiunte alla tabella creata precedentemente.
Analizziamo più dettagliatamente alcune istruzioni:

Per estrarre le informazioni riguardo alla risoluzione DNS, ad esempio, è stato utilizzato il tool tshark come segue:

tshark -r traffic.pcap -T fields -e frame.time_epoch -e frame.protocols -e dns.a -e dns.qry.name -Y "(dns.flags.response == 1)"

L'opzione ***-r traffic.pcap*** specifica il file da cui il tool tshark dovrà leggere. Le informazioni che si vogliono ricavare sono specificate dai campi che seguono l'opzione ***-T fields.***

Per il campo SNI abbiamo aggiunto al comando sopra riportato l'opzione:

-e tls.handshake.extensions_server_name

Mentre per il campo Host, partendo sempre da comando DNS, abbiamo aggiunto l'opzione:

-e http.host

Capitolo 4: Risultati sperimentali

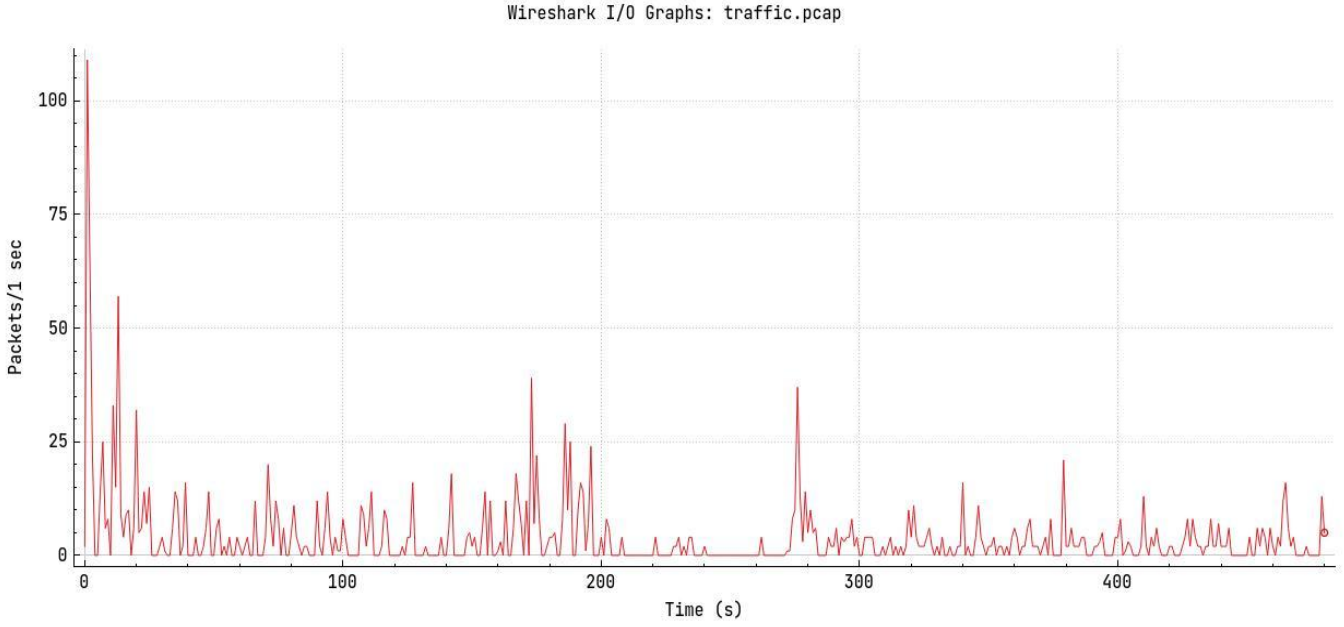
In questa sezione riportiamo i risultati ottenuti dalle analisi delle catture con alcune considerazioni. Vediamo due casistiche nello specifico.

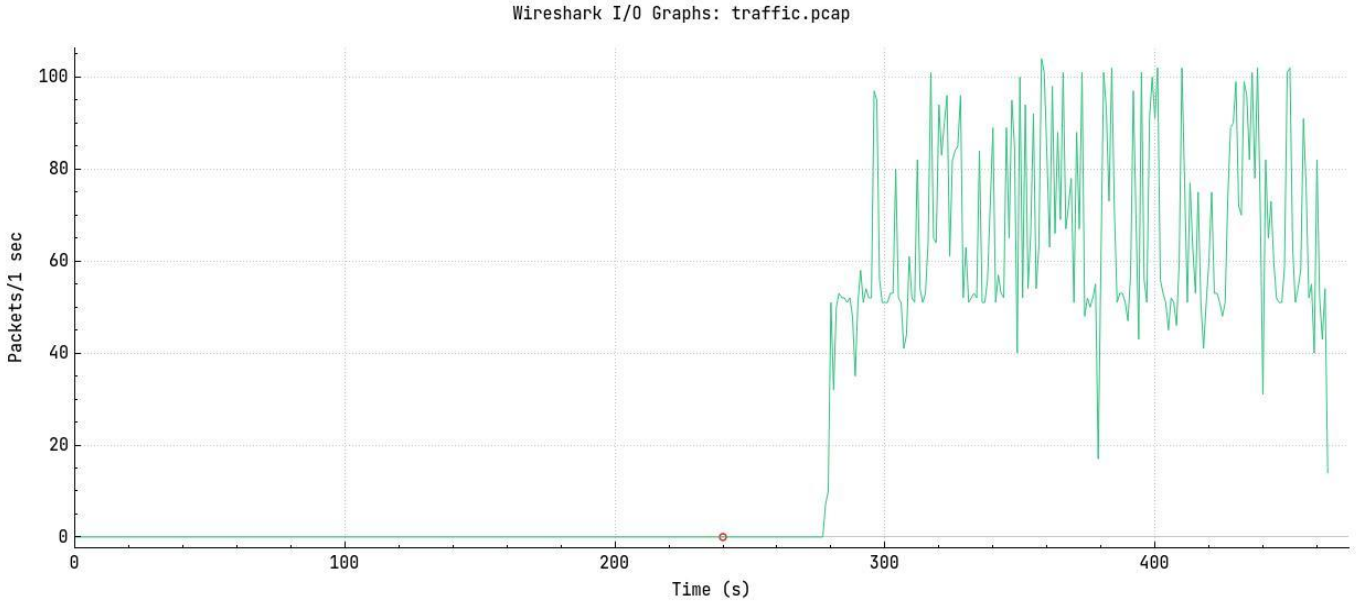
DISCORD (chat - audio call)

Usando il tool Capinfos, sono state estratte le seguenti informazioni di carattere generale. In particolare si evidenzia: Il numero di pacchetti, la dimensione media dei pacchetti, la velocità media di invio di pacchetti al secondo e il tempo totale di cattura, di circa 8 minuti.

```
File name:           traffic.pcap
File type:           Wireshark/tcpdump/... - pcap
File encapsulation:  Ethernet
File timestamp precision:  microseconds (6)
Packet size limit:   file hdr: 262144 bytes
Number of packets:   14 k
File size:           3
Data size:           3
Capture duration:    480.075307 seconds
First packet time:   2022-11-25 14:46:17.161322
Last packet time:    2022-11-25 14:54:17.236629
Data byte rate:      6
Data bit rate:       52 kbps
Average packet size: 222.70 bytes
Average packet rate: 29 packets/s
SHA256:              0e894abf0dcbd64c5b78ae791ad24e4dbe19b2c08de78e19b5105dd167e9708e
RIPEMD160:           76c43186007bf7cb1e0850a633fa2b67d3fafea7
SHA1:                6def29f2a7502feb7a4a6d60989c3044b93152d3
Strict time order:   True
Number of interfaces in file: 1
Interface #0 info:
                     Encapsulation = Ethernet (1 - ether)
                     Capture length = 262144
                     Time precision = microseconds (6)
                     Time ticks per second = 1000000
                     Number of stat entries = 0
                     Number of packets = 14131
```

Di seguito vengono riportati alcuni grafici ottenuti mediante Wireshark. In rosso possiamo vedere la quantità di pacchetti inviati tramite TCP durante tutto l'arco di tempo di cattura, in verde i pacchetti inviati tramite UDP.

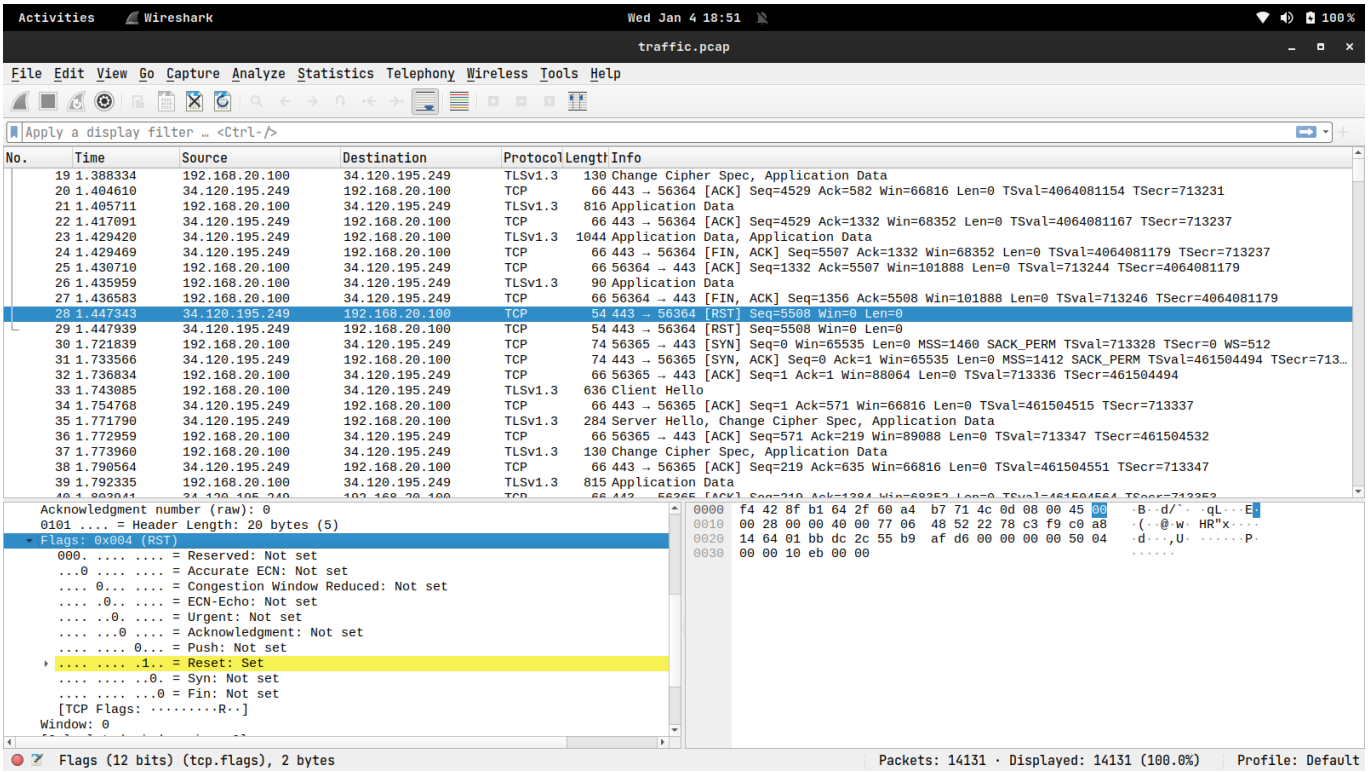




Durante l'attività di chat - audio call, eseguita con l'applicazione Discord, è interessante osservare la costante presenza di traffico TCP durante l'attività di chat testuale, come ci aspettavamo, data la necessità di avere garanzie circa l'avvenuta consegna dei messaggi.
TCP: Transmission Control Protocol è infatti un protocollo a livello di trasporto connection-oriented che, tramite i meccanismi di acknowledgement e di ritrasmissione, crea un servizio di trasferimento dati affidabile ovvero riesce a garantire la consegna dei dati senza errori ed in ordine.

Si osserva inoltre un picco di traffico UDP durante l'audio call. Tale attività, essendo time sensitive, ha bisogno di garanzie circa la velocità e il tempo con cui i pacchetti arrivano a destinazione. A tal proposito **UDP: User Datagram Protocol** è il protocollo a livello di trasporto adatto: è di tipo connectionless e non gestisce il riordinamento dei pacchetti né la ritrasmissione di quelli persi, in compenso è molto rapido e fornisce garanzie sul rispetto di una velocità minima (bassa latenza e overhead sulla rete molto minore di TCP). È quindi efficiente in contesti dove sono presenti processi real-time.

Notiamo infine un elevato numero di pacchetti inviati tramite TCP nell'intervallo (0-20)s. Ciò probabilmente è dovuto a diversi tentativi di creazione di una connessione stabile, non andati a buon fine, come possiamo vedere dalla figura riportata di seguito (che mostra una parte dei biflussi inviati tra client e server in dettaglio).
Alla riga 24-25 si possono osservare gli ultimi due step per la chiusura di una connessione TCP: il server invia un segmento con i flag ACK e FIN alzati e il client risponde con un segmento ACK. (Il primo step consistente nell'invio da parte del client di un segmento FIN è stato inviato in precedenza).



La riga evidenziata in blu mostra invece l'invio di un segmento TCP con il flag RST alzato. Tale segmento, privo di dati, è usato per effettuare il reset della connessione.
Subito dopo, dalla riga 30 alla riga 32, vi è infatti un esempio di three way handshake. Il client invia un segmento di controllo TCP SYN al server, questo riceve il SYN e risponde con un segmento SYN/ACK e infine il client dopo aver ricevuto correttamente quest'ultimo segmento risponde al server con un ACK.

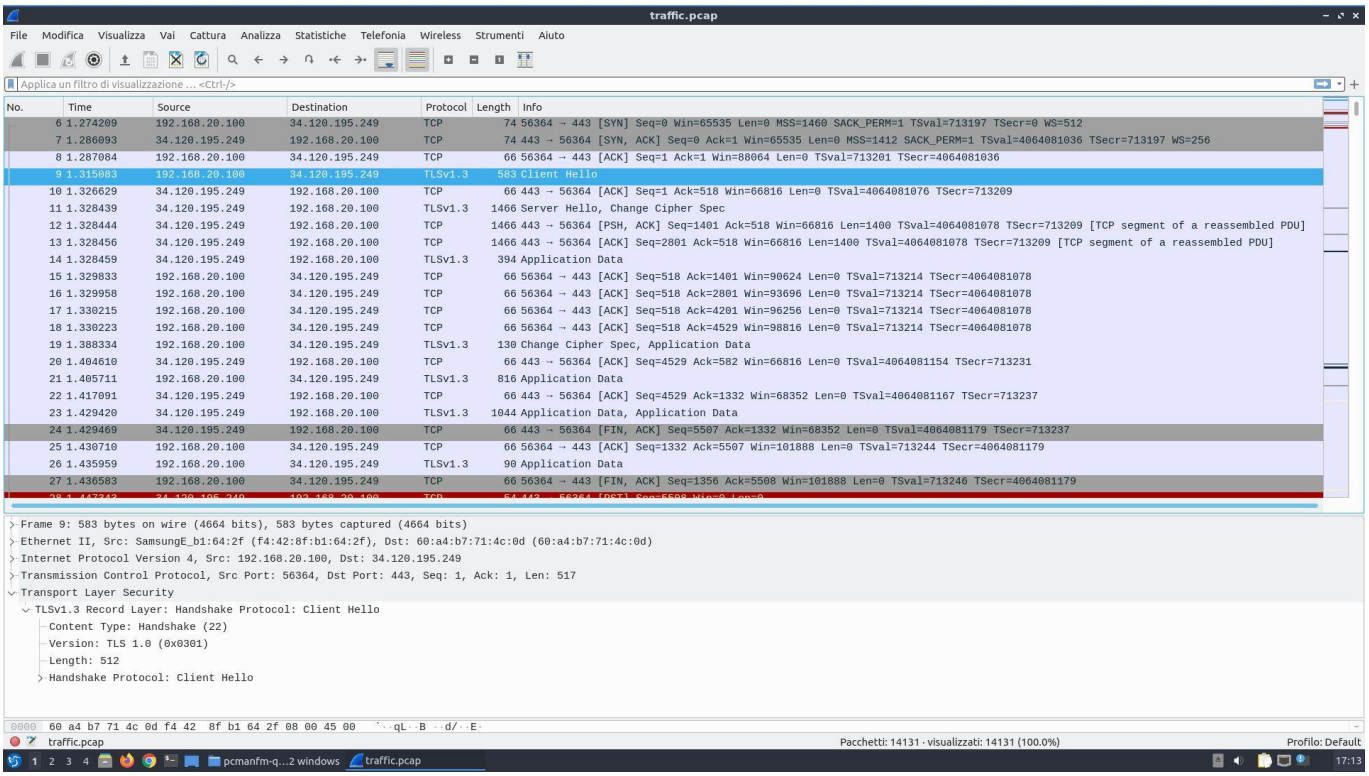
Procediamo con l'analisi del campo SNI del protocollo TLS.

TLS: Transport Layer Security è un protocollo che fornisce una connessione sicura per le applicazioni su reti TCP/IP. È un'evoluzione di SSL (Secure Socket Layer) e in particolare assicura autenticazione, integrità dei dati e confidenzialità operando al di sopra del livello di trasporto.
Per svolgere tali funzioni si fa uso di un handshake TLS. In questo processo client e server si scambiano informazioni prima ancora che inizi il trasferimento vero e proprio dei dati: si parte da una fase di autenticazione reciproca e infine si negoziano algoritmi di crittografia e le chiavi crittografiche per proteggere i dati. Se tali operazioni sono andate a buon fine, inizia la comunicazione e lo scambio di dati. Se la verifica ha esito negativo, non vengono scambiati ulteriori dati.
Durante il primo passo di un handshake TLS (fase di autenticazione caratterizzata dall'invio di due messaggi: "client_hello" da parte del client e "server_hello" da parte del server) ci sono molte probabilità che risponda il sito sbagliato (essendo l'IPv6 non ancora affermato come standard, gli indirizzi IP risultano ancora molto limitati e non tutti i domini possono pretendere di avere un indirizzo IP tutto per sé), che non invii il certificato corretto con il giusto nome dell'host, e che quindi non venga stabilita la connessione. Per cui è importante che il client comunichi al server a quale dominio (host) sta cercando di connettersi. Proprio per questo è stato introdotto l'**SNI: Server Name Indication.**
L'SNI è quindi un'estensione di TLS. Ampliando il TLS con il Server Name Indication, il protocollo di sicurezza mette a disposizione con l'handshake un ulteriore campo: sotto il campo ClientHelloExtension si trova quello opzionale ServerName. In questo campo il client (e di questo si fa carico automaticamente il browser) inserisce il nome dell'host con il quale vorrebbe collegarsi. In questo modo è assicurato che risponda l'host corretto.

Nell'output delle nostre analisi risultano effettivamente numerose comunicazioni SNI di nome dell'host. Riportiamo un esempio:

1669383978.476405000	***eth:ethertype:ip:tcp:tls***	***064374.ingest.sentry.io***
1669383978.904407000	***eth:ethertype:ip:tcp:tls***	***064374.ingest.sentry.io***
1669383979.020283000	***eth:ethertype:ip:tcp:tls***	***discord.com***
1669383979.914657000	***eth:ethertype:ip:tcp:tls***	***gateway.discord.gg***
1669383984.046285000	***eth:ethertype:ip:tcp:tls***	***discord.com***
1669383998.066164000	***eth:ethertype:ip:tcp:tls***	***latency.discord.media***
1669384150.273614000	***eth:ethertype:ip:tcp:tls***	***play.googleapis.com***
1669384150.273614000	***eth:ethertype:ip:tcp:tls***	***play.googleapis.com***

Alle righe 9 e 11, dopo un three way handshake, notiamo i messaggi che vengono scambiati durante la prima fase dell' handshake TLS.



Per quanto riguarda l'analisi del campo host di http, le uniche richieste HTTP che compaiono sono del tipo generate_204, richieste fatte in caso di riconnessione da parte di un cellulare Android, le risposte sono standardizzate con contenuto vuoto e codice di stato 204.

Ne riportiamo un' esempio:

1669392931.481829000 eth:ethertype:ip:tcp:http connectivitycheck.gstatic.com /generate_204 GET HTTP/1.1
No Content 204

Infine per le risoluzioni DNS il server contattato è stato sempre quello di Google, tra le richieste principali ricordiamo: risoluzioni dei nomi per lo specifico servizio che stavamo utilizzando (mtalk.google.com, gateway.discord.gg, ecc), per servizi di hosting e per l'indirizzo da contattare in caso di perdita di connessione da parte del cellulare Android (connectivitycheck.gstatic.com).

Di seguito sono mostrate le richieste DNS comparse durante la sessione di cattura di Discord.

1669393211.796688000 eth:ethertype:ip:udp:dns 142.250.180.164 www.google.com
1669393211.796824000 eth:ethertype:ip:udp:dns 142.250.180.131 connectivitycheck.gstatic.com

Dall'esecuzione del comando whois sono risultati i seguenti intestatari degli indirizzi IP:

ANALISI INDIRIZZI IP CON WHOIS
130.61.33.65:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

130.61.132.30:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

141.147.37.129:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

141.144.250.183:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

104.18.21.227:
OrgName: Cloudflare

8.8.4.4:
OrgName: Level 3 Parent
OrgName: Google LLC

193.122.15.164:

5.91.172.147:

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

130.61.132.30:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

141.147.37.129:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

Riportiamo qui di seguito una veloce descrizione delle organizzazioni proprietarie degli indirizzi IP:

Cloudflare Inc: è una società americana che si occupa di content delivery network, servizi di sicurezza internet e servizi di DNS distribuiti, che si pongono tra i visitatori di un sito e gli hosting provider degli utenti Cloudflare, agendo come un reverse proxy server per siti web.

Google LLC: è un'azienda informatica statunitense che offre servizi online, con quartier generale a Mountain View in California, nel cosiddetto Googleplex.

Level 3 Parent LLC: Level 3 Communications è una multinazionale americana che si occupa di telecomunicazioni e di internet service provider, la cui sede principale si trova a Broomfield, Colorado.
Level 3 opera come network di Tier 1. La compagnia si occupa del trasporto di contenuti (come Netflix e Apple music and video) in Nord America, America Latina, Europa e alcune città dell'Asia.

RIPE NCC: è il registro Internet regionale per l'Europa, il Medio Oriente e parti dell'Asia centrale. La sua sede è ad Amsterdam, nei Paesi Bassi, con una filiale a Dubai, negli Emirati Arabi Uniti. Un RIR sovrintende all' allocazione e alla registrazione delle risorse del numero Internet in una regione specifica.

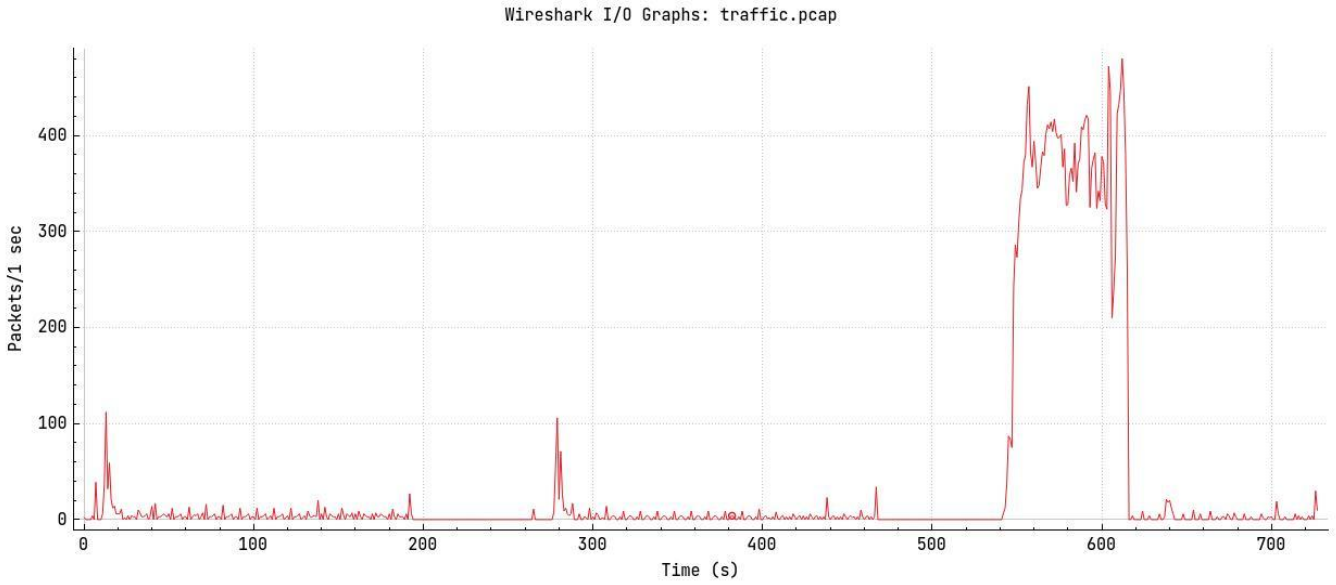
Oracle Corporation: è una società multinazionale del settore informatico, con sede a Austin, Texas. L'azienda vende software e tecnologia per database, sistemi di ingegneria cloud e prodotti software aziendali, in particolare i propri marchi di sistemi di gestione di database.

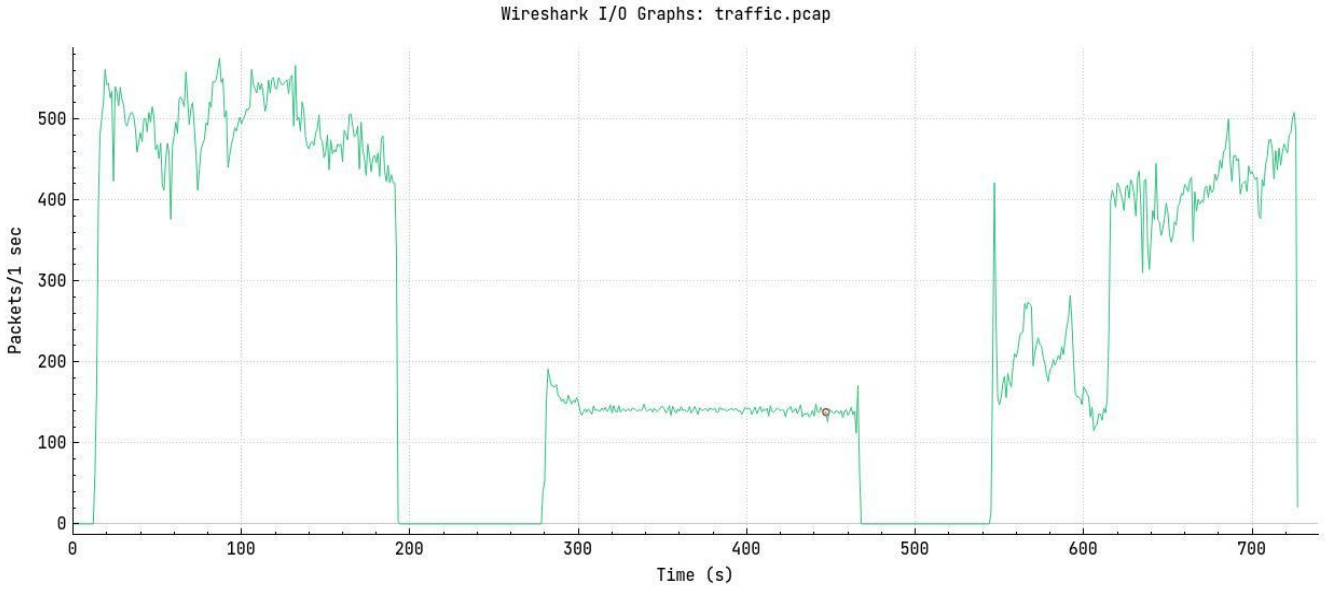
JITSI MEET (video call - audio call)

Come in precedenza, usando il tool Capinfos, sono state estratte le seguenti informazioni di carattere generale. In particolare si evidenzia: Il numero di pacchetti, la dimensione media dei pacchetti, la velocità media di invio di pacchetti al secondo e il tempo totale di cattura di circa 12 minuti.

```
File name:           traffic.pcap
File type:           Wireshark/tcpdump/... - pcap
File encapsulation:  Ethernet
File timestamp precision:  microseconds (6)
Packet size limit:   file hdr: 262144 bytes
Number of packets:   202 k
File size:           136 MB
Data size:           133 MB
Capture duration:    727.391169 seconds
First packet time:   2022-11-25 17:52:49.419754
Last packet time:    2022-11-25 18:04:56.810923
Data byte rate:      183 kBps
Data bit rate:       1
Average packet size: 659.06 bytes
Average packet rate: 277 packets/s
SHA256:              45bc4ce51c3c546d3faabc81a19d5a3c0e6070fe0de247b64541f3c6e0f78ddf
RIPEMD160:           9c6db54894d5df6f3fa1e30a7829d0e0b2551c65
SHA1:                bb957b7b6e4da14edbba42443664c8829b1ffec9
Strict time order:   False
Number of interfaces in file: 1
Interface #0 info:
                     Encapsulation = Ethernet (1 - ether)
                     Capture length = 262144
                     Time precision = microseconds (6)
                     Time ticks per second = 1000000
                     Number of stat entries = 0
                     Number of packets = 202104
```

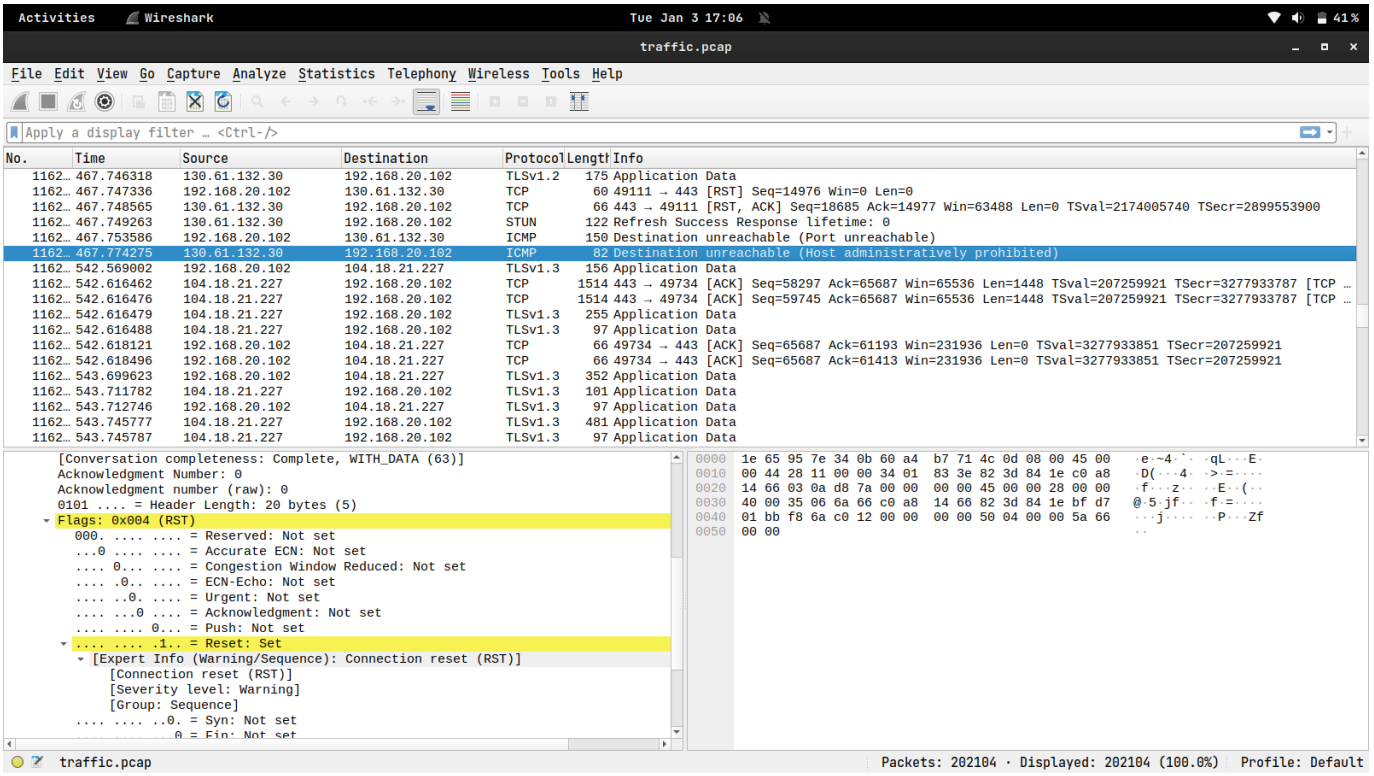
Anche qui riportiamo i due grafici dei protocolli: TCP in rosso, UDP in verde





Durante l'attività svolta su JitsiMeet possiamo notare quasi esclusivamente l'utilizzo del protocollo UDP anziché TCP, per i motivi detti nell'analisi precedentemente. (Eccezion fatta per i minuti di pausa dove notiamo assenza di pacchetti).

Interessante è la trasmissione di pacchetti tra i 540 s e i 610 s, dove si può notare un elevato picco di pacchetti mandati con TCP e al tempo stesso un calo drastico di pacchetti inviati con UDP. Proviamo a spiegare il perchè:



Dalla tabella riportata sopra, si evidenzia in blu l'utilizzo del protocollo ICMP, poco prima del picco.

ICMP: Internet Control Message Protocol è un protocollo che può essere usato per veicolare diversi tipi di messaggi di gestione, identificati primariamente dal tipo e dal relativo codice. In particolare qui si riporta un anomalia "Destination unreachable", identificata prima da un "port unreachable" (type 3, code 3) e successivamente da un "Host administratively prohibited" (type 3, code 10) probabilmente dovuto ad un errore a livello firewall.

Possiamo dedurre che ci sia quindi stato un drastico calo della qualità della connessione e che sia stato risolto tramite un utilizzo più intenso del protocollo TCP a discapito di UDP.

Dall’esecuzione del comando whois sono risultati i seguenti intestatari degli indirizzi IP:

ANALISI INDIRIZZI IP CON WHOIS
130.61.33.65:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

130.61.132.30:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

141.147.37.129:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

141.144.250.183:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

104.18.21.227:
OrgName: Cloudflare

8.8.4.4:
OrgName: Level 3 Parent
OrgName: Google LLC

193.122.15.164:

5.91.172.147:

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

130.61.132.30:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

141.147.37.129:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

130.61.132.30:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

5.91.172.147:

193.122.56.141:

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

141.144.250.183:
OrgName: RIPE Network Coordination Centre
org-name: Oracle Svenska AB

130.61.33.65:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

5.91.172.147:

193.122.15.164:

130.61.132.30:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

158.101.183.222:
OrgName: Oracle Corporation
OrgName: Oracle Public Cloud

Per la specifica delle organizzazioni proprietarie degli indirizzi IP si veda l’analisi precedente.

Con considerazioni analoghe alla sessione Dicord riportiamo il campo SNI di TLS (il campo host di HTTP e le richieste DNS sono identiche all'analisi precedente):

1669395176.677376000 eth:ethertype:ip:tcp:tls meet.jit.si
1669395182.598759000 eth:ethertype:ip:tcp:tls meet-jit-si-turnrelay.jitsi.net
1669395182.784004000 eth:ethertype:ip:tcp:tls
meet-jit-si-eu-frankfurt-1-local-3603-jvb-40-89-44.jitsi.net
1669395448.736004000 eth:ethertype:ip:tcp:tls meet-jit-si-turnrelay.jitsi.net
1669395448.918880000 eth:ethertype:ip:tcp:tls
meet-jit-si-eu-frankfurt-1-local-3603-jvb-40-105-90.jitsi.net
1669395714.474128000 eth:ethertype:ip:tcp:tls meet-jit-si-turnrelay.jitsi.net
1669395715.269377000 eth:ethertype:ip:tcp:tls
meet-jit-si-eu-frankfurt-1-local-3603-jvb-40-92-158.jitsi.net

Concludiamo con una descrizione dei protocolli, oltre a TCP, UDP e ICMP, che sono stati utilizzati durante tutte le sessioni di cattura.

RTP: Real-time Transport Protocol è un protocollo utilizzato per servizi di comunicazione in tempo reale su Internet come l'interattività audio e video. È basato sul protocollo UDP. Solitamente le applicazioni pongono l'RTP sopra l'UDP per le operazioni di multiplexing e checksum, anche se può essere usato con altri protocolli di rete e trasporto sottostanti.

RTCP: Real-time Transport Control Protocol è un protocollo che raccoglie statistiche sulla qualità del servizio del protocollo RTP e trasporta le informazioni riguardo ai partecipanti ad una sessione.

ARP: Address resolution protocol è un protocollo utilizzato per la conversione degli indirizzi IP a livello rete in indirizzi MAC a livello di collegamento.

STUN: Session Traversal Utilities for Network Address Translators (NATs) si tratta di un protocollo che permette alle applicazioni, principalmente applicazioni IP (ad esempio, i telefoni VoIP, Voice over IP) di scoprire la presenza ed i tipi di NAT e firewall che si interpongono tra il computer e la rete pubblica. STUN permette a queste applicazioni di conoscere gli indirizzi IP e le porte con cui il dispositivo NAT li sta rendendo visibili sulla rete pubblica.

DTLS: Datagram Transport Layer Security è un protocollo di comunicazione progettato per proteggere la privacy dei dati e prevenire intercettazioni e manomissioni. Rispetto a TLS che sfrutta TCP, DTLS invece sfrutta il protocollo di trasporto UDP.

QUIC: Quick UDP Internet Connections, presente durante le sessioni di Google Meet, è un protocollo di rete di livello di trasporto inizialmente progettato da Google. L'intenzione dietro a QUIC è il desiderio di sviluppare un'alternativa ai protocolli di sicurezza affermati e basati su TCP, ovvero HTTP/2 e TLS/SSL.
Sebbene lo standard HTTP Secure (o HTTPS) si dimostri un alleato affidabile nella protezione della privacy degli utenti, registra anche un rallentamento dei tempi di caricamento a causa dell'handshake TLS e dello scambio di chiavi e certificati.
QUIC invece avvia una connessione con un singolo pacchetto (due se si tratta di un primo contatto) e comunica tutti i parametri TLS o HTTPS necessari. Nella maggior parte dei casi un client può inviare i dati anche direttamente a un server, anche senza dover dipendere da una risposta, mentre il TCP deve per prima cosa ottenere ed elaborare una conferma del server.
Il risultato è una connessione quasi equivalente a TCP ma con una latenza notevolmente ridotta.

Conclusioni

Questo elaborato ha evidenziato come l'utilizzo di determinati protocolli sia fortemente legato alle esigenze delle applicazioni. In particolare abbiamo notato come con attività di chat venisse usato principalmente TCP mentre con attività di audio o video call UDP.
Abbiamo inoltre approfondito argomenti non trattati durante il corso di studi (si veda ad esempio l'uso, durante le comunicazioni client-server, di molti più protocolli di quelli studiati) e sviluppato conoscenze da un punto di vista più pratico.